Formal verification and reproducibility supplement

Companion to Paper 1 v2.7 and Paper 2 v1.6; revision 1.3 (GitHub archive edition); 1 November 2026

This supplement retains the detailed verification and provenance sections from Paper 1 v2.3 and Paper 2 v1.3. The earlier editorial revision changed no Lean source or mathematical theorem. The archived rev 5 and rev 6 evidence below retains its original scope; Section 6 records the separate rev 7 extension and its current verification status.

What is and is not certified. All three main rev-5 results have complete formal proofs and a successful independent project rebuild. The supplementary rigidity limit and exact Dai–Chen comparison are conventional paper proofs, not separate Lean theorems. The stronger $1675\sqrt{3}/1536$ instance has a separately checked proof, and Cedar's independent rev-6 replay verified its integration in the 39-module extension. The original rev-5 archive remains unchanged. The rev 7 source extension is described separately in Section 6.

Correspondence between the conventional theorems and the Lean statements was established by manual source review, supported by four separately checked bridge lemmas. Lean checks those lemmas and the formal statements; it does not machine-check the translation of this prose into Lean.

Citation authenticity. The accompanying 15-entry citation audit distinguishes direct primary records from corroboration in published bibliographies. Thirteen works were matched to an arXiv, publisher, or original author-archive record. The 1979 Gerver–Ramsey and 1999 Erdős–Lev–Rauzy–Sándor–Sárközy articles are independently corroborated in several primary papers' reference lists; their original publisher pages were not accessible in this audit. No fabricated reference was identified. This is not a claim that every cited proof has been reread or that literature priority is settled. In particular, Fang's full proof remains an access-dependent method-comparison task.

Independent test. Use the public rev 5 tree at `https://github.com/Dlgt-io/room-math-results/tree/main/results/erdos-271/lean/rev5`, not the earlier bootstrap kit. After checking its manifest, follow the archived `VERIFY.md` at `https://github.com/Dlgt-io/room-math-results/tree/main/results/erdos-271/lean/archived-extras` and run

```
export EXPECT_SUMS=\
2d4f22165d1e3b60461315f8cd4e2d8e70d0498aaf1add86e4bd459ff4f090ef
bash lean/rev5/scripts/replay_fresh.sh /path/to/new-empty-project
```

where the target directory must not already exist. The full command with the expected manifest hash and an optional pinned dependency donor is in the archive's guide. The acceptance criterion is a successful full build followed by the strict 24-name axiom audit, regression tests, source-token scan, bundle composition check, and final source-hash check. A fresh `logs/SUCCESS.json` is written only after all checks pass. Internet access may be needed to

install the pinned toolchain and dependencies; the proof sources do not depend on a research-room connection.

1 Paper 1: detailed formalization record

The main theorem has a complete Lean 4.35.0 formalization, including the standard-seed wrappers. The modular release pins Mathlib at commit

`56a005855ea17e0b0958f73ec801ae140b0a1679`

The following evidence concerns different objects and must not be conflated.

Layer	Evidence at this revision
Modular project	Author build, followed by Cedar's independent fresh-directory rebuild of all 38 rev-5 modules on 22 October 2026. The strict audit passed for all 24 expected declarations, with unchanged source hashes.
External checker	Seven self-contained proof bundles, B1/B5/B6/B7/B8a/B8b/B9, accepted in the fixed Lean/Mathlib environment. Their actual returned statements and input hashes were independently inspected.
Bundle composition	All six dependency links match. The exact accepted terms compile together as `ComposedC9.composed`, also rebuilt by Cedar. Separately, the checker accepted the implication from the seven bundle statements to C_9 (job `68fdc6fb-0c74-4be9-a243-8d7f6ad400ef`, archived in `extras/c9-glue/`; independently resubmitted as job `e0c9847a`). Together these are eight checker verdicts in one environment joined by modus ponens, not one kernel check: a single-term C_9 attempt (job `bb93cb63`) was rejected for exceeding the default `maxHeartbeats`.
Final deductions	Two separately written endgames compile over a shared analytic layer; these are not independent reconstructions of every dependency.
Statement fidelity	Source reviews cover the covering property, positive versus full counts, indexing and greedy cutoff. Four standalone bridge statements also passed the external checker.

The reported modular axiom dependencies are `propext`, `Classical.choice`, and `Quot.sound`. No `sorry`, added axiom, `native_decide`, or unsafe declaration is used in the reported source closure. Individual external acceptance does not itself constitute a local full project build or supply a new transitive axiom report.

For clarity, write S_i for the proposition proved by an unconditional checker bundle and C_i for the final conclusion of a conditional one. The verified interfaces are

$$B1 : S_1, \quad B5 : S_5, \quad B6 : S_6, \quad B7 : S_6 \to C_7,$$
$$B8a : S_1 \to S_5 \to C_7 \to C_{8a}, \quad B8b : C_{8a} \to C_8, \quad B9 : C_8 \to C_9.$$

Their compiled direct application is

$$B9(B8b(B8a\ B1\ B5\ (B7\ B6))).$$

Thus no analytic obligation remains unproved in this dependency graph. The interface check and the compilation of this application are separate checks. The compiled version uses the actual externally accepted proof terms, not axioms for their conclusions. Cedar subsequently rebuilt this composed declaration as part of the full frozen rev-5 project. This should not be confused with submitting one monolithic proof term to the external checker.

The unconditional formal assertion is the explicit witness theorem `Erdos271.IndependentEndgame.arbitrarily_large_witnesses`, equivalently `Erdos271.Final.erdos271_limsup_any` in eventual-envelope form. The separate real-valued `Filter.limsup` declaration `Erdos271.Final.erdos271_limsup_ge_any` assumes that the normalized quotient is bounded above on a tail; if it is not, the extended-real limsup in the paper is $+\infty$. The infinity of S follows from the covering hypothesis, rather than being an additional exported declaration. The standard two-seed consequences are `Erdos271.Stanley.two_seed_limsup` and `Erdos271.Stanley.two_seed_ak`. The finite identity is `Erdos271.FiniteIntegrals.finite_weighted_integral_identity`; its self-contained checker bundle B5 passed (job `2c4ac1f1`). It is built in the modular source, although it is not a separate name among the 24 pinned headline reports. A frozen-prefix definition with cutoff m is the usual $S(A_0)$ only when $m = \max A_0$; the standard wrappers impose this. The additional fidelity lemmas prove uniqueness of a continuation satisfying the greedy rule, identify the finite positive count, justify the natural-number subtraction in covering witnesses, and equate the no-envelope statement with arbitrarily large witnesses.

From paper lemmas to Lean files. The following is a guide to the main reusable statements, not a requirement to inspect the formal proof to understand the paper. The modules are part of the pinned release.

Conventional step	Principal Lean source and declaration
Weak covering, pair count, lower envelope	`Audit/WCover.lean`, `Audit/Moy.lean`
Weighted pair inequality	`Audit/MemberTail.lean`; checker bundle B1
Finite integral identity (Section 3)	`WeightedIdentity.lean`: `finite_weighted_integral_identity`; bundle B5
Kernel mass, moment and logarithmic window	`Audit/KMChecked.lean`, `Audit/Glue4K.lean`, `Audit/Glue5.lean`, `Audit/Glue6.lean`; bundles B6–B7
Contradiction and arbitrarily large witnesses	`Audit/Final.lean`: `limit_of_budget`; `Audit/General.lean`: `erdos271_limsup_any`; `IndependentEndgame.lean`: `arbitrarily_large_witnesses`
Enumeration and ordinary greedy seeds	`Audit/AkForm.lean`, `Audit/Stanley.lean`: `stanley_std_limsup`, `stanley_std_ak`
Rigidity's finite inequality	`IndependentEndgame.lean`: `fixed_error_budget`

The externally accepted bundles are composed by type-checked function application, with standard axioms. The frozen project, including that composition, now also has an independent fresh-directory rebuild. The paper's conditions on weak covering, positive versus full counts, natural subtraction, zero-indexed enumeration and the greedy continuation have also been compared to the checked statements. These comparisons are manual scope/fidelity reviews; they are separate from the compilation evidence and are not themselves machine-checked.

The defect-budget inequality retaining $\mathcal{R} + \kappa D$ is formalized by `Erdos271.IndependentEndgame.fixed_error_budget`.

The subsequent logarithmic-mean limiting argument in Paper 1, Corollary 6.1 and the exact Dai–Chen comparison are reviewed paper deductions, not separate completed Lean claims.

Independent rebuild and remaining review. Cedar's rev-5 replay completed at 19:46:23 UTC on 22 October 2026, with exit code zero, 38 freshly compiled project modules, 73 matching payload hashes, a strict 24-of-24 axiom audit, and 16 passing audit regression tests. The source manifest is `2d4f22165d1e3b60461315f8cd4e2d8e70d0498aaf1add86e4bd459ff4f090ef`.

The project was rebuilt from source using pinned cached dependencies; Lean and Mathlib themselves were not rebuilt from source. The report and raw evidence are in the public `verification/rev5/` directory at `https://github.com/Dlgt-io/room-math-results/tree/main/results/erdos-271/verification/rev5`. This evidence concerns rev 5, not rev 6 or a new literature-priority certificate.

This remains a public review draft, not a submission. Human authorship, a permanent public archive, a final literature-priority check and external expert review remain release tasks. The theorem comparison establishes improvement over Dai–Chen's cited result; absence of a later or unindexed equivalent theorem has not been exhaustively established.

2 Paper 2: detailed formalization record

This is a public review draft, not an externally submitted paper. The following is a status record for the release pinned to Lean 4.35.0 and Mathlib commit `56a00585`; the full commit hash is given in Section 1 and the release manifest below.

Evidence terminology. Here *external check* means a self-contained statement/proof term accepted by the fixed Lean/Mathlib checker. *Modular build* means a theorem compiled in the full pinned source tree in the author's environment. *Independent complete rebuild* means compiling that full source tree in a separate environment. These are distinct assurances and none is asserted without its own evidence.

The limsup theorem. The declarations `Erdos271.B16.B16_main` and `Erdos271.B16.B16_exists` include the recursive set definition, strong covering beyond 88, a uniform counting bound and arbitrarily large lower witnesses for every $U < 17/6$. Their self-contained statement/proof pairs were accepted by the fixed remote checker in jobs beginning `7f84be19` and `e343345f` (statement SHA-256 prefixes `cf6d045e` and `609db29c`). Both checker statements contain the uniform inequality and arbitrary-large-witness formulation, not the `Filter.limsup` equality. The defining equation is not an unsupported existence assumption: the existence statement is also checked. The modular development additionally contains `mem_iff_blocks`, identifying the recursive and block definitions, and `B16_limsup`/`S16_limsup`, giving the explicit `Filter.limsup` equality. Those additional declarations were also compiled in Cedar's fresh full-project rebuild; they are not separate standalone checker submissions.

The liminf theorem. `Erdos271.Liminf.StaticCert.liminf_of_cert` is the generic transfer theorem (Paper 2, Theorem 4.2); its 416-element instance is `Erdos271.Liminf.liminf_construction_1889`. These are compiled in the author's pinned modular project and Cedar's independent replay, with standard transitive axiom reports; they are not themselves standalone checker-submitted terms. The external checker separately accepted Claude's complete, self-contained proof of the same $419\sqrt{3}/384$ existence assertion in job `d5e58222` (statement SHA-256 prefix `186d2fca`). It constructs the same finite sets using symbolic lifts and bounded repair checks after deleting 13. Grok independently recompiled that precise proof term and obtained only `propext`, `Classical.choice` and `Quot.sound`. Its slightly looser count admits at most four instead of three future-stage points, which does not change the asymptotic constant. A different direct proof using S_{768} and constant $53\sqrt{3}/48$ was checker-verified as bundle B4 (job `0e2507a5`); it is not a remote check of `liminf_of_cert`. The newer $1675\sqrt{3}/1536$ result above has its own separate checker verdict and does not modify the frozen modular project.

Counting conventions. The base-16 formal count is $A(x) = |A_{16} \cap [1, \lfloor x \rfloor]|$, which omits zero, so $F_{A_{16}}(x) = A(x)+1$. Its inequality $A(x)+1 \le (17/6)\sqrt{x}$ is exactly the full-count bound in Paper 2, Theorem 1.1. The locally compiled declarations `B16_limsup` and `S16_limsup` state $\limsup_{x\to\infty} A(x)/\sqrt{x} = 17/6$ over real x; since $F_{A_{16}}(x)/\sqrt{x} = A(x)/\sqrt{x} + 1/\sqrt{x}$, they give the same limit superior. In contrast, the formal liminf results use an ε–N statement: one set B is chosen first, and for every $\varepsilon > 0$ and every $N \in \mathbb{N}$ there exists an integer $n \ge N$ with $|B \cap [0, n]| \le (c + \varepsilon)\sqrt{n}$. This implies the real-variable liminf in Paper 2, equation (1.3). The count includes any zero; no positivity hypothesis is needed for this implication, although positivity of the constructed set is established internally and in the conventional proof. Every bounded subset of $\mathbb{N}_0$ is finite, so `ncard` here equals ordinary finite cardinality.

What has and has not been rebuilt. The frozen 38-module release has a clean build and fresh-directory replay in the author's environment, pinned by the release `SHA256SUMS` file (SHA-256 `2d4f22165d1e3b60461315f8cd4e2d8e70d0498aaf1add86e4bd459ff4f090ef`). Only hashed source files were transferred into the fresh directory; no locally compiled project artifacts were reused. Its strict audit accepts exactly 24 named headline theorems, each appearing once, with only `propext`, `Classical.choice` and `Quot.sound`. An audit test suite includes duplicate/missing-name and unexpected-axiom rejection. Cedar completed the separate fresh-directory rebuild at 19:46:23 UTC on 22 October 2026. All 38 project modules were compiled, all 73 payload hashes matched before and after the run, the strict audit passed for 24 distinct expected names, and all 16 regression tests passed. The runner exited with code zero and produced `logs/SUCCESS.json`. It used pinned cached dependencies: Lean and Mathlib were not rebuilt from source. The report and raw logs are available under `verification/rev5/` in the public archive. This independently reproduced build concerns the frozen rev 5 project; it does not automatically certify the rev 6 additions, literature priority, or a fresh informal-to-formal review.

Navigating the Lean development. For a human reader, the recommended order is the main theorem, its two or three central lemmas, and only then the finite certificates. In the pinned release the corresponding statements are:

Paper step	Lean file / theorem
Base-16 definition and covering	`B16.lean`: `S16_char`, `mem_iff_blocks`, `S16_cover`, `B16_main`, `B16_exists`
Digit count and global estimate	`B16.lean`: `N_block`, `N_16m`, `encode`, `member_bound`, `bound_all`
Limsup equality	`B16Limsup.lean`: `B16_limsup`, `S16_limsup`
Finite lift and repair	`LiminfCore.lean`: `lift_cov`, `step_cov`, `iter_spec`
Generic finite-to-infinite theorem	`LiminfGen.lean`: `StaticCert.liminf_of_cert`
Frozen 416-element certificate	`LiminfCert49152.lean`: `S49152_cov`, `S49152_card`
Frozen liminf constant	`LiminfFinal.lean`: `liminf_construction_1889`

The manuscript proofs are independent of any familiarity with Lean. Finite witness lists and other generated data are isolated from the conceptual lemmas. A standalone checker term may be much longer than the corresponding modular proof because the external service does not accept project imports; that should not be mistaken for the intended reader-facing source architecture.

3 Contribution and literature-access record

The base-16 construction and finite static certificates originate in Claude's room notes; the complemented-digit counting argument was developed by ChatGPT; Grok assembled the con-

crete Lean constructions and the generic lift-and-stack formalization. These are contribution records for the collaboration, not a settled publication author list.

This draft freezes the 17/6 and $419\sqrt{3}/384$ targets. Earlier constructions are not advertised as additional release theorems. The smaller $1675\sqrt{3}/1536$ instance is now separately checker-verified, but lies outside the frozen 38-module source and is presented only as a clearly labeled remark. The independent frozen-project rebuild is complete. Before submission, the author list, final literature-priority review, permanent public source release and external specialist review remain to be settled. The comparison with Fang uses his publisher's precise theorem statement; his full subscription article was not available to the manuscript assembler, so the comparison of proof methods remains open. The full Kiss–Sándor–Yang and Chen preprints were consulted for definitions and comparison statements. A 2026 preprint of Saengrungkongka (arXiv:2609.02041) was reviewed by a collaborator; its main theorems concern $k \geq 6$ or higher-order covering/Kakeya questions, and no overlap with our three stated constants was found. This is not an exhaustive priority check for the current $k = 3$ bounds.

4 Why unordered integer saturation is not forward covering

This comparison concerns Csajbók–Nagy's integer construction (Theorem 4.13 of `arXiv:2401.06283v4`). Its finite-group results are not needed here. Let H_l contain the length-l base-4 integers with digits in $\{2, 3\}$, for $l \geq 1$, and put $S = \bigcup_{l \geq 1} H_l$. Define

$$x_l = \frac{2(4^{l+1} - 1)}{3} - 1, \qquad N_l = |S \cap [1, x_l]| = 2^{l+1} - 2.$$

There are $x_l - N_l$ non-members in the prefix. A forward witness $a < b < n \leq x_l$ uses a pair from that same prefix, and each pair represents only one target. Thus the number of non-members without forward witnesses is at least

$$x_l - N_l - \binom{N_l}{2} = \frac{2 \cdot 4^l + 9 \cdot 2^l - 8}{3}.$$

Dividing by x_l gives a lower bound tending to $1/4$. This occurs along these cutoffs; it is not an assertion about every individual digit block. Deleting a fixed initial interval cannot remove all these failures. Hence the unordered saturation property of this set does not give eventual weak forward covering. The argument is a conventional counting proof, not a new Lean result or a claim of novel saturation theory.

5 Identity of the separately checked stronger instance

The full checker job for $1675\sqrt{3}/1536$ is
`042a1cf7-e251-4071-90f5-b1f384498329`.

Its statement SHA-256 is
`f34ea2a577ba67182a137abd724314a1680934c5b5e52e9886a1145a90589836`
and its proof SHA-256 is
`4cab2175f543161e3fff9819fb22bf70aa39920232f83be2bfbec39009ef81b3`.

The adopted rev-6 manifest is
`78291fa4ca02d51370c4b2c56073ef58bb5f98d7553a7fa77c5fc156d2219c03`.

Cedar's independent replay completed at 12:42:29 UTC on 23 October 2026, with exit code zero, 77 matching payload hashes, 39 freshly compiled project modules, 26 distinct audited declarations, and all 17 audit regression tests passed with the real transcript. The printed types and axiom reports for the original 24 declarations were byte-identical

to the accepted rev-5 run. The receipts are `verification/rev6/SUCCESS.json` and `verification/comparison-and-acceptance.json` in the public archive at `https://github.com/Dlgt-io/room-math-results/tree/main/results/erdos-271/verification`. The October 2026 archive edition inspected those receipts; that editorial round did not perform another Lean build. The replay used pinned cached dependencies, not a Lean/Mathlib source rebuild. Rev 5 and its successful verification remain unchanged.

6 Rev 7 challenge and solution comparator

Rev 7 is a GitHub-only extension of the weighted-covering formalization. It preserves the rev 6 manuscript and Lean sources and introduces no new mathematical bound or research-room adoption. `Statement.lean` defines the inclusive counting function $F_S(x) = |S \cap [0, x]|$, returning zero when $x < 0$, and defines weak AP_3-covering by explicit witnesses $a < b < z$ with $z = 2b - a$. `Challenge.lean` imports `Statement.lean` alone; the statement module depends on Mathlib, not on the solution or legacy proof modules.

The challenge theorem is `Erdos271.main_result`. It asserts the abstract's lower bound $\sqrt{2\sqrt{2}/\log(1+\sqrt{2})}$ for the extended-real limsup of $F_S(x)/\sqrt{x}$ under weak covering, with no eventual-boundedness assumption and no `DecidablePred` hypothesis. `Solution.lean` imports `Abstract.lean` and the legacy proof modules to prove this theorem. `Abstract.lean` supplies the bridge from the legacy positive count and real-valued endgame to the inclusive count and extended-real conclusion; the theorem's mathematical content is unchanged.

Comparator protocol and status. On 1 November 2026, native `lake comparator` from Lean 4.35.0 accepted `Erdos271.main_result`. It compared the trusted challenge with the solution, checked the permitted axioms, and replayed the exported proof using both Lean's kernel and the independent `nanoda` kernel. The configuration permits only `propext`, `Quot.sound`, and `Classical.choice`. Controls accepted a valid proof and rejected a changed theorem statement and an extra axiom.

The checking environment was prepared from a clean Linux runtime and a fresh official Lean release. Pinned Mathlib dependencies and their public cache were fetched afresh. Comparator ran before any ordinary proof build; no project build outputs were copied from an earlier run. Later project build and audit commands run in a nonnetwork bubblewrap sandbox. Project sources, checker binaries, and parent-owned evidence are read-only, while the project's `.lake` build directory and temporary scratch space are writable. The sandbox controls check non-root execution, zero effective and permitted capabilities, no-new-privileges, read-only evidence, and absence of an external network route.

The scripts record raw comparator output, standard error, exit code, tool versions, and source and checker hashes under `lean/rev7/logs/comparator/`, outside the proof-writable `.lake` directory. The archived evidence and combined replay receipt are under `verification/rev7/`. The complete replay also builds the inherited constructions and audits 27 designated declarations. The earlier rev 6 replay and standalone checker receipts retain their historical scope. Cached dependencies were used; Lean and Mathlib were not rebuilt from source. The trusted statement and referenced definitions still require mathematical review.